Análisis: Lab 3 – Programación Orientada a Objetos

Daniel Eduardo Sacol Cojón – 26870

1. Requisitos funcionales

Información que el sistema debe conservar:

- Inventario de envíos: Código único, remitente, destinatario, peso real, destino (urbano o departamental), estado actual y cantidad de intentos realizados.
- Flotilla de vehículos: Placa única, capacidad máxima en kilogramos, si cuenta con refrigeración y si está actualmente en uso.
- Despachos: Vehículo asignado, lista de envíos cargados, estado del despacho y peso total cargado.
- Finanzas: Acumuladores de ingresos totales separados por registros iniciales y por recargos de reintentos.

Operaciones que debe permitir:

- Cotizar envíos y registrar nuevos envíos.
- Registrar nuevos vehículos para la flotilla.
- Crear despachos, asignarles envíos en bodega y retirar envíos.
- Dar salida a un despacho hacia su ruta y posteriormente registrar los resultados (entregado o fallido) para cada envío.
- Consultar inventarios, vehículos y generar reportes gerenciales de ingresos y estados.

Condiciones de validez de cada operación:

- No se permiten valores numéricos negativos, fuera de rango, ni códigos o placas repetidos o vacíos.
- Los sobres no pueden exceder 0.5 kg de peso.
- La carga refrigerada debe tener una temperatura requerida entre $-25°C$ y $8°C$, y su destino debe ser obligatoriamente urbano.
- Para asignar un envío a un despacho: el envío no puede estar en otro despacho, la suma de pesos reales no debe superar la capacidad del vehículo, y las cargas refrigeradas solo pueden ir en vehículos con refrigeración.
- No puede salir a ruta un despacho vacío.

2. Decisiones que dependen de la categoría

Las decisiones que varían según el tipo de envío son:

- El cálculo del costo de envío.
- El cálculo del tiempo estimado de entrega.
- El límite máximo de intentos de entrega permitidos.

Quién toma estas decisiones y por qué:

Cada clase hija (Sobre, Paquete, PaqueteFragil y CargaRefrigerada) toma sus propias decisiones. La clase padre Envio define estos métodos como abstractos, lo que obliga a las clases hijas a implementar sus reglas específicas. Esto se hace gracias al polimorfismo, lo cual permite que el sistema principal le pida a cualquier envío que calcule su costo o tiempo sin importar qué categoría sea, evitando tener que usar condicionales largos (if/else) para preguntar el tipo de objeto.

3. Estados y transiciones

Estados de un Envío:

- En bodega: Es el estado inicial al registrar el envío. También regresa a este estado si el personal lo retira de un despacho en preparación, o si falla un intento de entrega y aún le quedan intentos disponibles.
- En ruta: Cambia a este estado cuando el despacho al que fue asignado sale de la bodega.
- Entregado: Cambia a este estado cuando el despacho regresa y el personal registra la entrega como exitosa.
- Devuelto al remitente: Cambia a este estado cuando la entrega falla y el envío agota su límite máximo de intentos permitidos.

Estados de un Despacho:

- En preparación: Es el estado inicial cuando se elige un vehículo y se le empiezan a asignar envíos. Aquí se pueden agregar o retirar paquetes.
- En ruta: Ocurre cuando el personal hace "salir" el despacho. Ya no se pueden modificar sus envíos.
- Cerrado: Se alcanza cuando el despacho regresa y todos sus envíos tienen un resultado final registrado. En este momento el vehículo queda libre de nuevo.

4. Clases, métodos y atributos necesarios

1. Tabla de Clases

Clase	Tipo / Capa	Responsabilidad
Envio	Abstracta / Modelo	Define la información base, los estados y firma el contrato polimórfico (costos, tiempos, intentos) para todos los envíos.
Sobre	Concreta / Modelo	Representa documentos ligeros (hasta 0.5 kg) con tarifas y tiempos de entrega fijos.
Paquete	Concreta / Modelo	Representa envíos volumétricos. Calcula su peso facturable y aplica costos dinámicos por kilogramo.
PaqueteFragil	Concreta / Modelo	Hereda de Paquete. Añade reglas de embalaje, cálculo de seguro sobre el valor declarado y un día adicional de tránsito.
CargaRefrigerada	Concreta / Modelo	Representa envíos con cadena de frío. Controla rango de temperatura permitida ($-25°C$ a $8°C$) y restricción de destino.
Vehiculo	Concreta / Modelo	Almacena los datos de un vehículo de la flotilla, su capacidad en peso real y si es apto para cadena de frío.
Despacho	Concreta / Modelo	Agrupa envíos asignados a un vehículo. Valida límites de capacidad, compatibilidad térmica y control de estados de ruta.
Envios	Concreta / Modelo	Fachada principal de la lógica. Administra el inventario, la flotilla, el historial de despachos y consolida los ingresos.
VistaConsola	Concreta / Vista	Maneja toda la entrada y salida de texto. Muestra menús, captura datos sin interrumpirse por errores de formato y muestra reportes.
ControladorEnvios	Concreta / Controlador	Orquesta la comunicación. Pide datos a la Vista, invoca las validaciones del Modelo y muestra mensajes de éxito o rechazo.

2. Tabla de Atributos

Clase	Visibilidad	Tipo	Nombre	Propósito / Regla
Envio	- (private)	String	codigo	Identificador único del envío.
Envio	- (private)	String	remitente	Nombre de quien envía.
Envio	- (private)	String	destinatario	Nombre de quien recibe.
Envio	- (private)	double	pesoReal	Masa física en kilogramos.
Envio	- (private)	boolean	esUrbano	true si es en Ciudad de Guatemala, false departamental.

Envio	- (private)	String	estado	Estado actual: en bodega, en ruta, entregado o devuelto.
Envio	- (private)	int	intentos	Cantidad de intentos de entrega realizados.
Paquete	- (private)	double	largo	Longitud en centímetros.
Paquete	- (private)	double	ancho	Anchura en centímetros.
Paquete	- (private)	double	alto	Altura en centímetros.
PaqueteFragil	- (private)	double	valorDeclarado	Valor en quetzales para cálculo del 3% de seguro.
CargaRefrigerada	- (private)	double	temperaturaRequerida	Temperatura en grados Celsius (entre $-25°C$ y $8°C$).
Vehiculo	- (private)	String	placa	Identificador único del vehículo.
Vehiculo	- (private)	double	capacidadMax	Peso límite soportado en kilogramos reales.
Vehiculo	- (private)	boolean	refrigerado	true si cuenta con equipo para cadena de frío.
Vehiculo	- (private)	boolean	enUso	true si tiene un despacho activo sin cerrar.
Despacho	- (private)	Vehiculo	vehiculo	Referencia al vehículo asignado para la ruta.
Despacho	- (private)	List<Envio>	envios	Lista de referencias a los envíos cargados.
Despacho	- (private)	String	estado	Estado actual: en preparación, en ruta, cerrado.
Despacho	- (private)	double	pesoActual	Sumatoria de los pesos reales de los envíos asignados.
Envios	- (private)	List<Envio>	inventario	Almacenamiento en memoria de todos los envíos registrados.
Envios	- (private)	List<Vehiculo>	flotilla	Almacenamiento en memoria de todos los vehículos registrados.
Envios	- (private)	List<Despacho>	despachos	Registro histórico y activo de despachos.
Envios	- (private)	double	ingresosRegistro	Acumulador de dinero por registro inicial de envíos.
Envios	- (private)	double	ingresosReintentos	Acumulador de recargos de Q15 por intentos adicionales.
VistaConsola	- (private)	Scanner	scanner	Lector de entradas del teclado.
ControladorEnvios	- (private)	Envios	modelo	Referencia a la capa de datos (ahora de tipo Envios).
ControladorEnvios	- (private)	VistaConsola	vista	Referencia a la interfaz.

3. Tabla de Métodos

Clase	Visibilidad	Retorno	Método y Parámetros	Propósito / Regla
Envio	+ (public)	double	calcularCosto()	**Abstracto.** Exige implementar cálculo de tarifa.
Envio	+ (public)	int	calcularTiempo()	**Abstracto.** Exige implementar días estimados.
Envio	+ (public)	int	getLimiteIntentos()	**Abstracto.** Exige implementar la cantidad máxima de intentos.
Envio	+ (public)	void	registrarIntento(boolean exito)	Actualiza estado y suma intentos.
Envio	+ (public)	*Varios*	getAtributo() / setAtributo(Tipo a)	Métodos de acceso estándar para atributos comunes.
Sobre	+ (public)	double	calcularCosto()	Retorna Q20 (urbano) o Q35 (departamental).
Sobre	+ (public)	int	calcularTiempo()	Retorna 1 o 2 días según destino.
Sobre	+ (public)	int	getLimiteIntentos()	Retorna límite estricto de 3.
Paquete	+ (public)	double	calcularPesoVolumetrico()	Retorna: $largo \times ancho \times alto / 5000$.
Paquete	+ (public)	double	calcularCosto()	Calcula con peso mayor (real vs volumétrico) más base (Q25/Q40) + Q8/kg.
Paquete	+ (public)	int	calcularTiempo()	Retorna 2 o 4 días según destino.
Paquete	+ (public)	int	getLimiteIntentos()	Retorna límite estricto de 3.
Paquete	+ (public)	*Varios*	getDimension() / setDimension(double d)	Métodos de acceso para largo, ancho y alto.

PaqueteFragil	+ (public)	double	calcularCosto()	Llama a costo de paquete base y suma Q15 + 3% del valor declarado.
PaqueteFragil	+ (public)	int	calcularTiempo()	Llama a tiempo base y le suma 1 día extra.
PaqueteFragil	+ (public)	int	getLimiteIntentos()	Retorna límite estricto de 2.
PaqueteFragil	+ (public)	double / void	getValorDeclarado() / setValorDeclarado(double v)	Acceso al monto asegurado.
CargaRefrigerada	+ (public)	double	calcularCosto()	Tarifa base + Q12/kg real + Q60 fijos.
CargaRefrigerada	+ (public)	int	calcularTiempo()	Retorna siempre 1 día.
CargaRefrigerada	+ (public)	int	getLimiteIntentos()	Retorna límite estricto de 1.
CargaRefrigerada	+ (public)	double / void	getTemperatura() / setTemperatura(double t)	Acceso a la temperatura.
Vehiculo	+ (public)	*Varios*	getAtributo() / setAtributo(Tipo a)	Acceso a capacidad, placa, uso y soporte térmico.
Despacho	+ (public)	boolean	agregarEnvio(Envio e)	Valida límites (peso real, estado del envío, termodinámica) antes de asignar.
Despacho	+ (public)	boolean	retirarEnvio(Envio e)	Quita un envío y lo devuelve al estado "en bodega".
Despacho	+ (public)	void	iniciarRuta()	Cambia envíos a "en ruta". Dispara cobro de Q15 a envíos con > 1 intento previo.
Despacho	+ (public)	void	cerrar()	Libera el vehículo si todos los envíos cargados tienen resultado final registrado.
Envios	+ (public)	Envio	buscarEnvio(String codigo)	Retorna envío si existe en memoria, nulo si no.
Envios	+ (public)	void	registrarEnvio(Envio e)	Añade al inventario. Cobra y acumula en ingresosRegistro si el cliente acepta.
Envios	+ (public)	Despacho	crearDespacho(Vehiculo v)	Inicializa objeto despacho si el vehículo no tiene otro abierto.
Envios	+ (public)	void	registrarCargoExtra()	Suma Q15 al acumulador ingresosReintentos.
Envios	+ (public)	*Varios*	obtenerReportes()	Extrae conteos y sumatorias para vistas gerenciales.
VistaConsola	+ (public)	int	mostrarMenuPrincipal()	Imprime las opciones, lee teclado y retorna la opción.
VistaConsola	+ (public)	Map<String, Object>	solicitarDatosEnvio()	Pide paso a paso los datos iterando para evitar cierres por errores de tipo (ej. letras en vez de números).
VistaConsola	+ (public)	Map<String, Object>	solicitarDatosVehiculo()	Pide placa y capacidad al usuario.
VistaConsola	+ (public)	void	mostrarMensaje(String m)	Imprime advertencias, rechazos de operaciones o confirmaciones.
VistaConsola	+ (public)	void	mostrarReporte(String r)	Da formato de salida a los datos de la gerencia.
ControladorEnvios	+ (public)	void	iniciar()	Contiene el bucle do-while del menú y precarga los 8 envíos y 3 vehículos iniciales.
ControladorEnvios	- (private)	void	procesarCotizacionYRegistro()	Llama a pedir datos, calcula y si el usuario da OK, pasa el objeto a la clase Envios.
ControladorEnvios	- (private)	void	procesarRegistroVehiculo()	Gestiona el alta de flotilla.
ControladorEnvios	- (private)	void	procesarGestionDespachos()	Menú interno para crear, cargar, retirar o dar salida a despachos.
ControladorEnvios	- (private)	void	procesarResultadosEntrega()	Pregunta éxito o fracaso por cada envío del despacho y procesa reintentos.
ControladorEnvios	- (private)	void	generarReportesGerenciales()	Pide la data al modelo y ordena a la vista imprimirla.

5. Justificación: Relación entre Paquete y Paquete Frágil

Se propone que la clase Paquete Frágil herede directamente de la clase Paquete porque un paquete frágil es, en esencia, un paquete estándar que comparte exactamente la misma base lógica para calcular el peso volumétrico y las tarifas por kilogramo o fracción. Al aplicar esta herencia, se obtiene una gran reutilización de código, ya que para el método de cobro del paquete frágil solo se necesita llamar al costo base definido en su clase padre y sumarle los recargos específicos de embalaje y el 3% de seguro. Si se optara por una alternativa diferente, como hacer que el Paquete Frágil heredara directamente de la clase abstracta Envío, se perdería esta ventaja de diseño y el sistema se vería obligado a duplicar toda la lógica matemática de las dimensiones y el peso facturable en ambas clases.

6. Agregar una nueva categoría mercantil

Si la empresa decide agregar una nueva categoría, como envíos de mascotas o carga sobredimensionada, se tendría que crear una nueva clase específica que herede de la clase principal Envío. Dentro de esta nueva clase, sería necesario implementar las reglas correspondientes para calcular el costo, el tiempo estimado de entrega y la política de reintentos. Además, se requeriría una pequeña actualización en las clases de la Vista y el Controlador para ofrecer esta nueva opción en el menú de registro. En cuanto al impacto general, los reportes gerenciales no tendrían que cambiar en absoluto, ya que el sistema actual utiliza polimorfismo para recolectar los ingresos totales sin importar de qué categoría provengan. La clase Despacho tampoco sufriría modificaciones mayores, a menos que la nueva categoría traiga restricciones específicas, como que las mascotas solo puedan asignarse a cierto tipo de vehículo; en ese escenario puntual, únicamente se agregaría una condición adicional de validación en la clase Despacho antes de permitir la asignación del envío al vehículo.